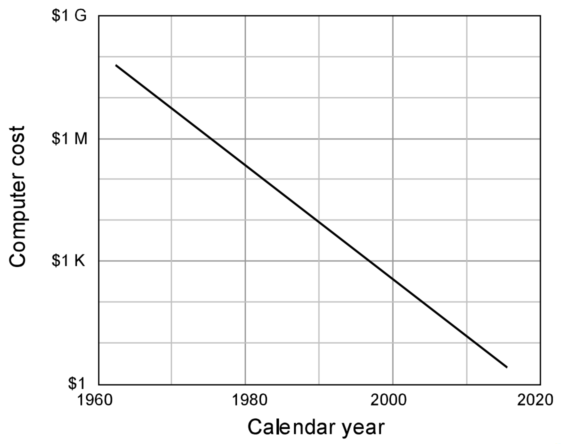
Computer Architecture

Computer Performance

Jaehyun Park (jhyun@inha.ac.kr)
School of Information and Communication
Inha University.

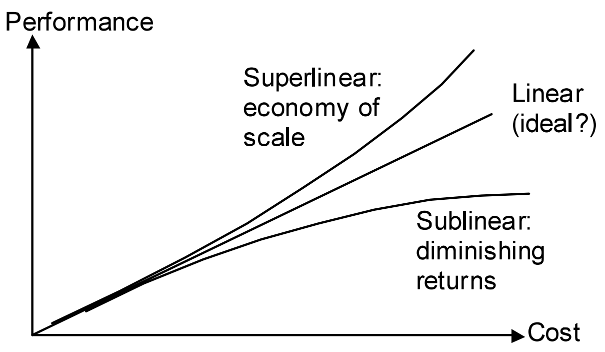
Embedded Computing Laboratory
School of Information & Communication

The Vanishing Computer Cost



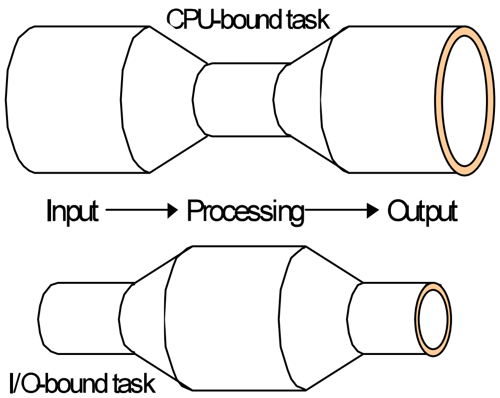
Embedded Computing Laboratory
School of Information & Communication

Cost/Performance



Embedded Computing Laboratory
School of Information & Communication

Defining Computer Performance



Embedded Computing Laboratory
School of Information & Communication

Cost, Performance, and Cost/Performance

Aircraft	Passengers	Range (km)	Speed (km/h)	Price ($M)
Airbus A310	250	8 300	895	120
Boeing 747	470	6 700	980	200
Boeing 767	250	12 300	885	120
Boeing 777	375	7 450	980	180
Concorde	130	6 400	2 200	350
DC-8-50	145	14 000	875	80

Different Views of performance

Performance from the viewpoint of a passenger: Speed

Note, however, that flight time is but one part of total travel time. Also, if the travel distance exceeds the range of a faster plane, a slower plane may be better due to not needing a refueling stop

Performance from the viewpoint of an airline: Throughput

Measured in passenger-km per hour (relevant if ticket price were proportional to distance traveled, which in reality is not)

Airbus A310 $250 \times 895 = 0.224$ M passenger-km/hr
Boeing 747 $470 \times 980 = 0.461$ M passenger-km/hr
Boeing 767 $250 \times 885 = 0.221$ M passenger-km/hr
Boeing 777 $375 \times 980 = 0.368$ M passenger-km/hr
Concorde $130 \times 2200 = 0.286$ M passenger-km/hr
DC-8-50 $145 \times 875 = 0.127$ M passenger-km/hr

Performance from the viewpoint of FAA: Safety

Cost Effectiveness: Cost/Performance

Aircraft	Passen-gers	Range (km)	Speed (km/h)	Price ($M)	Throughput (M P km/hr) — Larger values better	Cost / Performance — Smaller values better
A310	250	8 300	895	120	0.224	536
B 747	470	6 700	980	200	0.461	434
B 767	250	12 300	885	120	0.221	543
B 777	375	7 450	980	180	0.368	489
Concorde	130	6 400	2 200	350	0.286	1224
DC-8-50	145	14 000	875	80	0.127	630

Concepts of Performance and Speedup

Performance = 1 / Execution time   is simplified to

Performance = 1 / CPU execution time

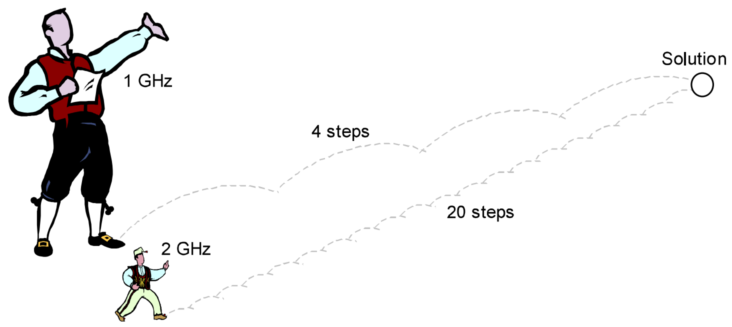
(Performance of M_1) / (Performance of M_2) = Speedup of M_1 over M_2
= (Execution time of M_2) / (Execution time M_1)

Terminology: M_1 is x times M_2 (e.g., 1.5 times as fast)
M_1 is $100(x - 1)\%$ M_2 (e.g., 50% faster)

CPU time = Instructions $\times$ (Cycles per instruction) $\times$ (Secs per cycle)
= Instructions $\times$ CPI / (Clock rate)

Instruction count, CPI, and clock rate are not completely independent, so improving one by a given factor may not lead to overall execution time improvement by the same factor.

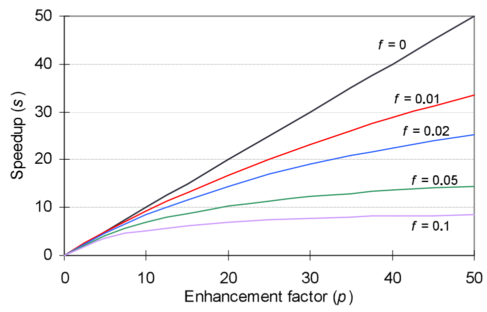
Faster Clock ≠ Shorter Running Time



Amdahl's Law

- Gene Amdahl [AMDA67]
- Deals with the potential speedup of a program using multiple processors compared to a single processor
- Illustrates the problems facing industry in the development of multi-core machines
  - Software must be adapted to a highly parallel execution environment to exploit the power of parallel processing
- Can be generalized to evaluate and design technical improvement in a computer system

Performance Enhancement: Amdahl's Law



f = fraction unaffected
p = speedup factor

$$s = \frac{1}{f + (1-f)/p}$$

$$\le min(p, 1/f)$$

Amdahl's law: speedup achieved if a fraction *f* of a task is unaffected and the remaining 1 – *f* part runs *p* times as fast.

Amdahl's Law Used in Design (Example 4.1)

A processor spends 30% of its time on flp addition, 25% on flp mult, and 10% on flp division. Evaluate the following enhancements, each costing the same to implement:

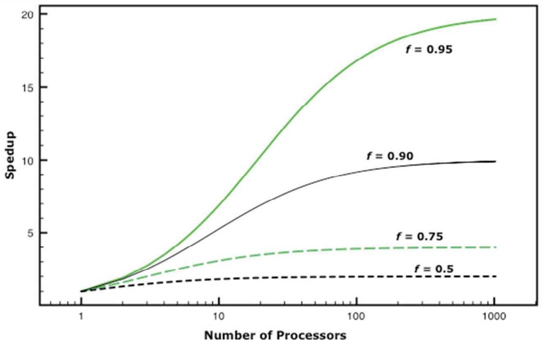
a. Redesign of the flp adder to make it twice as fast.
b. Redesign of the flp multiplier to make it three times as fast.
c. Redesign the flp divider to make it 10 times as fast.

Solution

a. Adder redesign speedup = 1 / [0.7 + 0.3 / 2] = 1.18
b. Multiplier redesign speedup = 1 / [0.75 + 0.25 / 3] = 1.20
c. Divider redesign speedup = 1 / [0.9 + 0.1 / 10] = 1.10

What if both the adder and the multiplier are redesigned?

Amdahl's Law

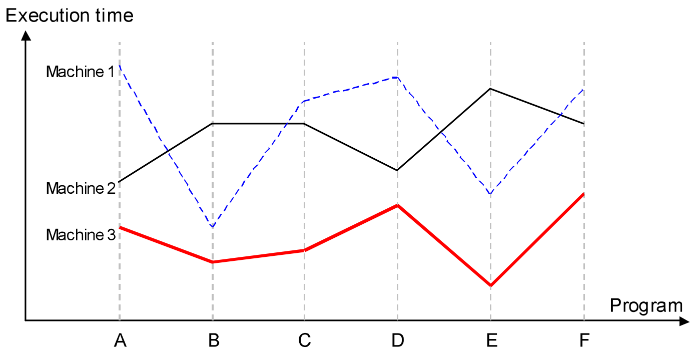


- Deals with the potential speedup of a program using multiple processors compared to a single processor
- Illustrates the problems facing industry in the development of multi-core machines
  - Software must be adapted to a highly parallel execution environment to exploit the power of parallel processing
- Can be generalized to evaluate and design technical improvement in a computer system

Little's Law

- Fundamental and simple relation with broad applications
- Can be applied to almost any system that is statistically in steady state, and in which there is no leakage
- Queuing system
  - If server is idle an item is served immediately, otherwise an arriving item joins a queue
  - There can be a single queue for a single server or for multiple servers, or multiples queues with one being for each of multiple servers
- Average number of items in a queuing system equals the average rate at which items arrive multiplied by the time that an item spends in the system
  - Relationship requires very few assumptions
  - Because of its simplicity and generality it is extremely useful

Performance Measurement vs. Modeling



Performance Benchmarks (Example 4.3)

You are an engineer at Outtel, a start-up aspiring to compete with Intel via its new processor design that outperforms the latest Intel processor by a factor of 2.5 on floating-point instructions. This level of performance was achieved by design compromises that led to a 20% increase in the execution time of all other instructions. You are in charge of choosing benchmarks that would showcase Outtel's performance edge.

a. What is the minimum required fraction f of time spent on floating-point instructions in a program on the Intel processor to show a speedup of 2 or better for Outtel?

Solution

a. We use a generalized form of Amdahl's formula in which a fraction f is speeded up by a given factor (2.5) and the rest is slowed down by another factor (1.2): $1 / [1.2(1 - f) + f / 2.5] \geq 2$ $\Rightarrow f \geq 0.875$

Performance Estimation

Average CPI = $\sum_{\text{All instruction classes}}$ (Class-i fraction) $\times$ (Class-i CPI)

Machine cycle time = 1 / Clock rate

CPU execution time = Instructions $\times$ (Average CPI) / (Clock rate)

Instruction Usage Frequency

Application → Instr'n class ↓	Data compression	C language compiler	Reactor simulation	Atomic motion modeling
A: Load/Store	25	37	32	37
B: Integer	32	28	17	5
C: Shift/Logic	16	13	2	1
D: Float	0	0	34	42
E: Branch	19	13	9	10
F: All others	8	9	6	4

MIPS Rating Can Be Misleading (Example 4.5)

Two compilers produce machine code for a program on a machine with two classes of instructions. Here are the number of instructions:

Class	CPI	Compiler 1	Compiler 2
A	1	600M	400M
B	2	400M	400M

a. What are run times of the two programs with a 1 GHz clock?
b. Which compiler produces faster code and by what factor?
c. Which compiler's output runs at a higher MIPS rate?

Solution

a. Running time 1 = (600M $\times$ 1 + 400M $\times$ 2) / 10^9 = 1.4 s
Running time 2 = (400M $\times$ 1 + 400M $\times$ 2) / 10^9 = 1.2 s
b. Compiler 2's output runs 1.4 / 1.2 = 1.17 times as fast
c. CPI of Compiler 1 = (600M $\times$ 1 + 400M $\times$ 2)/1000M = 1.4
1000/1.4 = 714 MIPS at 1GHz Clock
CPI of Compiler 2 = (400M $\times$ 1 + 400M $\times$ 2)/1000M = 1.5
1000/1.5 = 667 MIPS at 1GHz Clock

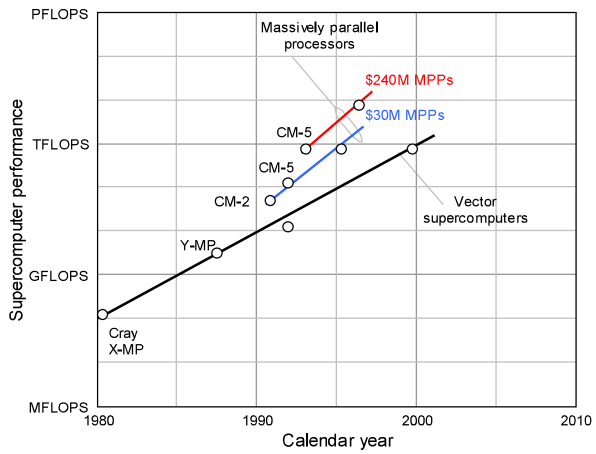
Reporting Computer Performance

	Time on machine X	Time on machine Y	Speedup of Y over X
Program A	20	200	0.1
Program B	1000	100	10.0
Program C	1500	150	10.0
All 3 prog's	2520	450	5.6

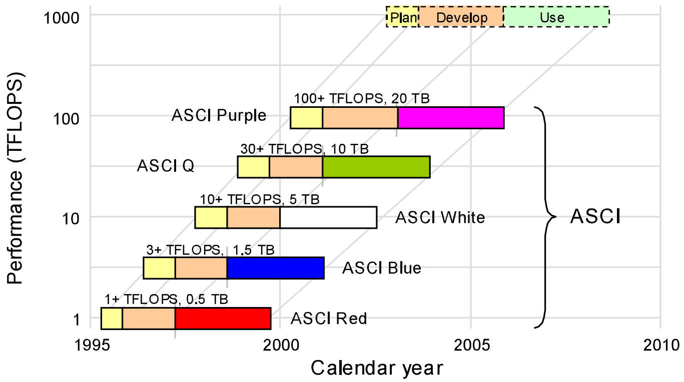
Not (0.1+10+10)/3=6.7

Analogy: If a car is driven to a city 100 km away at 100 km/hr and returns at 50 km/hr, the average speed is not (100 + 50) / 2 but is obtained from the fact that it travels 200 km in 3 hours.

Super-computers

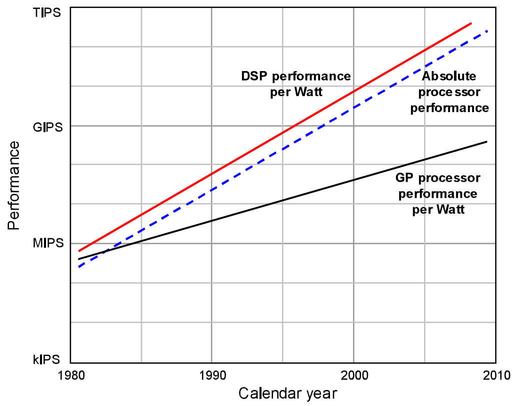


The Most Powerful Computers



Milestones in the DOE's Accelerated Strategic Computing Initiative (ASCI) program with extrapolation up to the PFLOPS level.

Trend in energy consumption per MIPS



Performance is Important, But It Isn't Everything

Benchmarks

For example, consider this high-level language statement:

A = B + C /* assume all quantities in main memory */

With a traditional instruction set architecture, referred to as a complex instruction set computer (CISC), this instruction can be compiled into one processor instruction:

```
add   mem(B), mem(C), mem (A)
```

On a typical RISC machine, the compilation would look something like this:

```
load  mem(B), reg(1);
load  mem(C), reg(2);
add   reg(1), reg(2), reg(3);
store reg(3), mem (A)
```


System Performance Evaluation Corporation (SPEC)

- Benchmark suite
  - A collection of programs, defined in a high-level language
  - Attempts to provide a representative test of a computer in a particular application or system programming area
- SPEC
  - An industry consortium
  - Defines and maintains the best known collection of benchmark suites
  - Performance measurements are widely used for comparison and research purposes
  - Best known SPEC benchmark suite
- SPEC CPU2006
  - Industry standard suite for processor intensive applications
  - Appropriate for measuring performance for applications that spend most of their time doing computation rather than I/O
  - Consists of 17 floating point programs written in C, C++, and Fortran and 12 integer programs written in C and C++
  - Suite contains over 3 million lines of code